

Índice general

Resumen	I
1. Introducción general	1
1.1. Domótica en motorhomes	1
1.2. Motivaciones del trabajo	1
1.3. Estado del arte	2
1.4. Objetivos y alcances	4
2. Introducción específica	7
2.1. Arquitectura general del sistema	7
2.2. Protocolos de comunicación	8
2.2.1. Protocolo CAN	8
2.2.2. Protocolo Modbus TCP	8
2.2.3. Solicitudes HTTP	9
2.3. Componentes de hardware	9
2.3.1. Microcontrolador ESP32	9
2.3.2. Módulo transceptor	10
2.3.3. Módulo Ethernet W5100	11
2.4. Herramientas de software	11
2.4.1. Entorno de desarrollo Arduino	11
2.4.2. Funcionamiento del ESP32	12
2.4.3. Desarrollo de la aplicación móvil	12
3. Diseño e implementación	15
3.1. Desarrollo de hardware	15
3.1.1. Comunicación con la red Enertik	16
3.1.2. Comunicación con la red Victron	18
3.1.3. Desarrollo de PCB final y listado de componentes	19
3.2. Desarrollo de firmware	20
3.2.1. Tarea: recepción UART	21
3.2.2. Tarea: decodificación de comando CAN	21
3.2.3. Tarea: comunicación Modbus TCP	22
3.2.4. Tarea: pedido de datos	23
3.2.5. Tarea: procesamiento de comandos recibidos	24
3.2.6. Tarea: control de servidor	24
3.3. Desarrollo de la aplicación	25
3.3.1. Arquitectura funcional de la aplicación	26
3.3.2. Secuencia de interacción entre la aplicación y el servidor	27
3.3.3. Generación de instalador para Android	28
4. Ensayos y resultados	31
4.1. Diseño final de la aplicación	31
4.2. Ensayos sobre la aplicación	32

Índice general

Resumen	I
1. Introducción general	1
1.1. Domótica en motorhomes	1
1.2. Motivaciones del trabajo	1
1.3. Estado del arte	2
1.4. Objetivos y alcances	4
2. Introducción específica	7
2.1. Arquitectura general del sistema	7
2.2. Protocolos de comunicación	8
2.2.1. Protocolo CAN	8
2.2.2. Protocolo Modbus TCP	8
2.2.3. Solicitudes HTTP	9
2.3. Componentes de hardware	9
2.3.1. Microcontrolador ESP32	9
2.3.2. Módulo transceptor	10
2.3.3. Módulo Ethernet W5100	11
2.4. Herramientas de software	11
2.4.1. Entorno de desarrollo Arduino	11
2.4.2. Funcionamiento del ESP32	12
2.4.3. Desarrollo de la aplicación móvil	12
3. Diseño e implementación	15
3.1. Desarrollo de hardware	15
3.1.1. Comunicación con la red Enertik	16
3.1.2. Comunicación con la red Victron	18
3.1.3. Prototipo experimental	19
3.1.4. Desarrollo de PCB final y listado de componentes	20
3.2. Desarrollo de firmware	22
3.2.1. Tarea: recepción UART	23
3.2.2. Tarea: decodificación de comando CAN	23
3.2.3. Tarea: comunicación Modbus TCP	24
3.2.4. Tarea: pedido de datos	25
3.2.5. Tarea: procesamiento de comandos recibidos	25
3.2.6. Tarea: control de servidor	26
3.3. Desarrollo de la aplicación	27
3.3.1. Aplicación: pestaña principal	28
3.3.2. Aplicación: pestaña luces	29
3.3.3. Aplicación: pestaña motores	30
3.3.4. Aplicación: pestaña energía	31
3.3.5. Aplicación: generación de instalador para Android	32

4.3. Ensayos sobre el sistema 33

4.4. Ensayos de laboratorio 34

 4.4.1. Prototipo experimental 35

 4.4.2. Ensayos de integración del sistema completo 36

5. Conclusiones 39

 5.1. Conclusiones generales 39

 5.2. Próximos pasos 40

Bibliografía 41

4. Ensayos y resultados 35

 4.1. Ensayos sobre la aplicación 35

 4.2. Ensayos sobre el sistema 36

 4.3. Ensayos de laboratorio 38

5. Conclusiones 43

 5.1. Conclusiones generales 43

 5.2. Próximos pasos 44

Bibliografía 45

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control.	3
2.1. Arquitectura general del sistema.	7
2.2. Microcontrolador ESP32 ¹	10
2.3. Funcionamiento del módulo transceptor.	10
2.4. Módulo W5100 ²	11
2.5. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.	13
3.1. Esquema de interconexión entre el ESP32 y los módulos externos.	15
3.2. Módulo transceptor comercial perteneciente a Enertik.	16
3.3. Diagrama de cómo se establece la comunicación CAN entre la red del motorhome y el ESP32.	17
3.4. Diseño de divisor resistivo para adaptación UART.	17
3.5. Diseño final para PCB del equipo.	20
3.6. Relación entre las distintas tareas del firmware y su comunicación con el exterior.	21
3.7. Contenedores dentro de una pantalla.	26
3.8. Arquitectura funcional de la aplicación.	27
3.9. Secuencia de solicitudes GET entre la aplicación y el microcontrolador.	28
3.10. Secuencia de solicitudes POST entre la aplicación y el microcontrolador.	28
4.1. Diseño final de las principales pantallas de la aplicación móvil.	31
4.2. Entorno de ejecución y depuración de la aplicación.	32
4.3. Archivo JSON recibido en la pestaña home.	33
4.4. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.	34
4.5. Respuesta del servidor ante una solicitud de control.	34
4.6. Prototipo experimental del equipo.	35
4.7. Módulo PRC10 utilizado en los ensayos.	36
4.8. Equipos Enertik utilizados en los ensayos.	37
4.9. Equipo Victron MultiPlus-II y módulo Cerbo GX.	38

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control.	3
2.1. Arquitectura general del sistema.	7
2.2. Microcontrolador ESP32 ¹	10
2.3. Funcionamiento del módulo transceptor.	10
2.4. Módulo W5100 ²	11
2.5. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.	13
3.1. Esquema de interconexión entre el ESP32 y los módulos externos.	15
3.2. Módulo transceptor comercial perteneciente a Enertik.	16
3.3. Diagrama de cómo se establece la comunicación CAN entre la red del motorhome y el ESP32.	17
3.4. Diseño de divisor resistivo para adaptación UART.	17
3.5. Prototipo experimental del equipo.	19
3.6. Diseño final para PCB del equipo.	21
3.7. Diagrama final de los componentes de hardware empleados.	22
3.8. Relación entre las distintas tareas del firmware y su comunicación con el exterior.	23
3.9. Contenedores dentro de una pantalla.	28
3.10. Pestaña principal de la aplicación.	28
3.11. Pestaña luces de la aplicación.	30
3.12. Pestaña motores de la aplicación.	31
3.13. Pestaña energía de la aplicación.	32
4.1. Entorno de ejecución y depuración de la aplicación.	35
4.2. Archivo JSON recibido en la pestaña home.	37
4.3. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.	37
4.4. Respuesta del servidor ante una solicitud de control.	38
4.5. Módulo PRC10 utilizado en los ensayos.	38
4.6. Equipos Enertik utilizados en los ensayos.	39
4.7. Equipo Victron MultiPlus-II y módulo Cerbo GX.	40

Para lograr la comunicación, resta definir qué dispositivo cumplirá el rol de maestro y cuál el de esclavo. Esta definición se abordará en la sección 3.2, donde se detallará su implementación.

3.1.3. Desarrollo de PCB final y listado de componentes

El diseño del PCB final se desarrolló mediante el software Altium Designer [31]. Este software es ampliamente utilizado en la industria, ya que permite crear esquemáticos de circuitos de forma sencilla. Estos sirven como base para el diseño de placas de circuito impreso.

Para la creación de este PCB, se tomó como referencia el conexionado utilizado en el prototipo experimental.

Dado que se espera que este desarrollo se convierta en un producto comercial, fue necesario realizar modificaciones en la selección de componentes durante el diseño de este prototipo.

La primera diferencia se encuentra en el microcontrolador ESP32. Para el prototipo experimental y los ensayos iniciales se utilizó una placa de desarrollo comercial basada en este dispositivo.

En el diseño del PCB final, se optó por reemplazar dicha placa por el chip del microcontrolador en sí. Esta decisión permitió descartar los componentes adicionales presentes en la placa de desarrollo. De esta forma, se eliminó hardware innecesario, se redujeron costos y se optimizó el espacio disponible en el PCB.

Por otra parte, fue necesaria la incorporación de un regulador de tensión AP63203 [32]. Este dispositivo permite adaptar el nivel de tensión de 12 V proveniente de la red del motorhome a un valor de 3,3 V, compatible con el chip ESP32.

En el caso del módulo transceptor, dado que se conocía completamente su diseño, se optó por integrarlo directamente en el PCB. Para ello, se incorporaron el microcontrolador PIC18F26K80 y el transceptor MCP2551 en la placa. Esta decisión permitió optimizar el espacio disponible y reducir costos al unificar todos los componentes en una única placa.

En cuanto al módulo W5100, se optó por utilizar la versión comercial disponible en el mercado. Como consecuencia, en el PCB se reservó un espacio para la colocación de un conector compatible con el módulo W5100, lo que permite su montaje manual de forma sencilla.

En la figura 3.5 se puede observar el diseño final del PCB.

Como se puede apreciar, el PCB está compuesto por los siguientes componentes:

- Componente 1: conector principal. Terminal sobre el que se conecta el cableado de cuatro hilos proveniente del motorhome.
- Componente 2: MCP2551. Forma parte del módulo transceptor, encargado de adaptar los niveles de tensión de la línea CAN.
- Componente 3: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceptor.

Para lograr la comunicación, resta definir qué dispositivo cumplirá el rol de maestro y cuál el de esclavo. Esta definición se abordará en la sección 3.2, donde se detallará su implementación.

3.1.3. Prototipo experimental

Para realizar los primeros ensayos, se decidió realizar un primer prototipo del equipo. Para ello, se utilizó como base un PCB (*Printed Circuit Board*) perforado de tipo experimental, sobre el que se soldaron diversos conectores, cableados y componentes de forma manual.

La figura 3.5 muestra el resultado del montaje del prototipo utilizado para los ensayos iniciales.

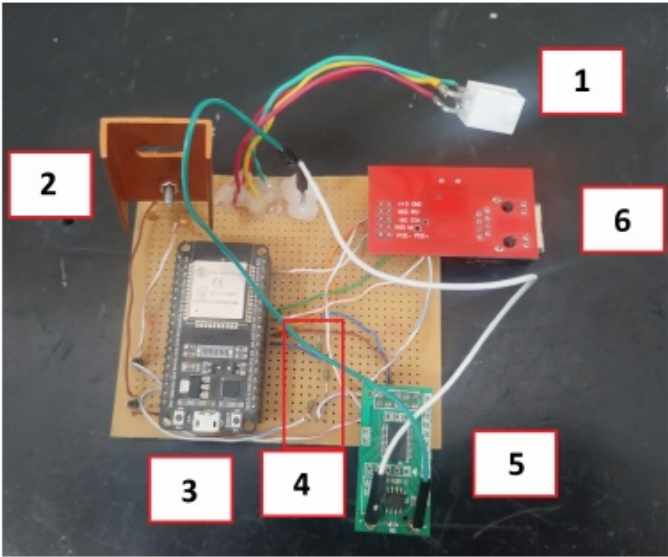


FIGURA 3.5. Prototipo experimental del equipo.

Como se puede apreciar, el prototipo cuenta con cada uno de los módulos explicados en la sección 3.1, donde:

- Componente 1: conector de cuatro hilos. Este conector es el estándar empleado en la red de domótica de Enertik. A través de él se energizan todos los componentes del PCB experimental y se accede a la comunicación CAN de la red.
- Componente 2: regulador de tensión 7805 [30]. Este dispositivo se encarga de suministrar una tensión estable de 5 V. Dado que la tensión proveniente del motorhome es de 12 V, fue necesaria su utilización para adecuar los niveles de alimentación a los requeridos por los módulos.

- Componente 4: PIC18F26K80. Microcontrolador que contiene la biblioteca desarrollada por Enertik para la comunicación entre equipos.
- Componente 5: módulo ESP32. Microcontrolador que contiene el firmware principal del equipo.
- Componente 6: regulador de tensión AP63203. Fuente de tensión de 3,3 V encargado de alimentar el ESP32.
- Componente 7: regulador de tensión 7805. Componente responsable de suministrar una tensión estable de 5 V para alimentar el PIC y el W5100.
- Componente 8: conector de diez pines. Conector compatible con el módulo W5100. En este lugar se montará dicho componente.

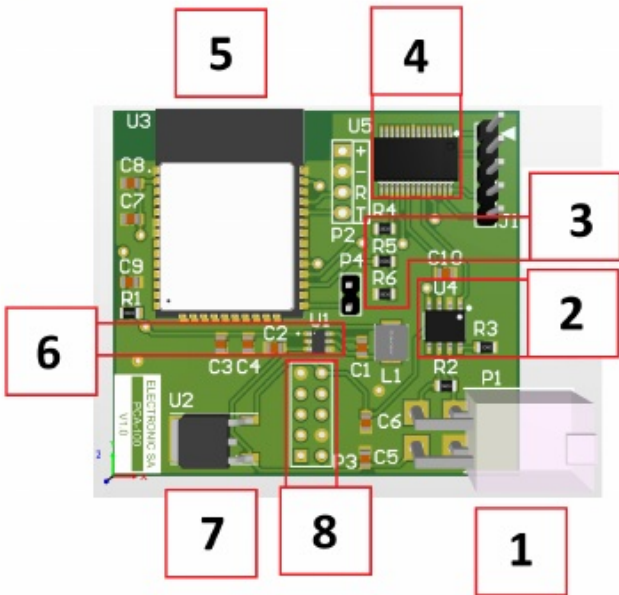


FIGURA 3.5. Diseño final para PCB del equipo.

Para el proceso de fabricación, fue necesaria la contratación de servicios de un fabricante especializado en este campo. Para ello, se generaron los denominados archivos Gerber [33].

3.2. Desarrollo de firmware

Como se mencionó en la sección 2.4.2, se optó por utilizar FreeRTOS como base para el desarrollo del firmware.

- Componente 3: módulo ESP32. Es el microcontrolador principal de la placa y contiene el firmware de control del sistema. En este prototipo se alimentó a través del regulador 7805, ya que la placa de desarrollo dispone internamente de reguladores que reducen la tensión a 3,3 V. Para el desarrollo final fue necesaria la incorporación de un regulador adicional dedicado al ESP32.
- Componente 4: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceptor.
- Componente 5: módulo transceptor. Se trata de una placa comercial provista por Enertik. En este prototipo se decidió incorporar un conector en el PCB experimental para facilitar su montaje y desmontaje de forma segura.
- Componente 6: módulo W5100. De manera similar al módulo transceptor, se utilizó un conector compatible que permite su montaje y desmontaje de forma sencilla y segura.

Este prototipo permitió validar el funcionamiento general del sistema y realizar pruebas iniciales de integración entre los distintos módulos de hardware.

3.1.4. Desarrollo de PCB final y listado de componentes

El diseño del PCB final se desarrolló mediante el software Altium Designer [31]. Este software es ampliamente utilizado en la industria, ya que permite crear esquemáticos de circuitos de forma sencilla. Estos sirven como base para el diseño de placas de circuito impreso.

Para la creación de este PCB, se tomó como referencia el conexionado utilizado en el prototipo experimental.

Dado que se espera que este desarrollo se convierta en un producto comercial, fue necesario realizar modificaciones en la selección de componentes durante el diseño de este prototipo.

La primera diferencia se encuentra en el microcontrolador ESP32. Para el prototipo experimental y los ensayos iniciales se utilizó una placa de desarrollo comercial basada en este dispositivo.

En el diseño del PCB final, se optó por reemplazar dicha placa por el chip del microcontrolador en sí. Esta decisión permitió descartar los componentes adicionales presentes en la placa de desarrollo. De esta forma, se eliminó hardware innecesario, se redujeron costos y se optimizó el espacio disponible en el PCB.

Por otra parte, fue necesaria la incorporación de un regulador de tensión AP63203 [32]. Este dispositivo permite adaptar el nivel de tensión de 12 V proveniente de la red del motorhome a un valor de 3,3 V, compatible con el chip ESP32.

En el caso del módulo transceptor, dado que se conocía completamente su diseño, se optó por integrarlo directamente en el PCB. Para ello, se incorporaron el microcontrolador PIC18F26K80 y el transceptor MCP2551 en la placa. Esta decisión permitió optimizar el espacio disponible y reducir costos al unificar todos los componentes en una única placa.

FreeRTOS permite estructurar el firmware mediante la ejecución de múltiples tareas. Su planificación genera un comportamiento concurrente a nivel lógico, lo que mejora la organización del sistema y facilita la gestión de sus funcionalidades.

En este contexto, para el presente trabajo se implementaron seis tareas principales, ilustradas en la figura 3.6, donde se aprecian las relaciones existentes entre ellas y con los módulos externos.

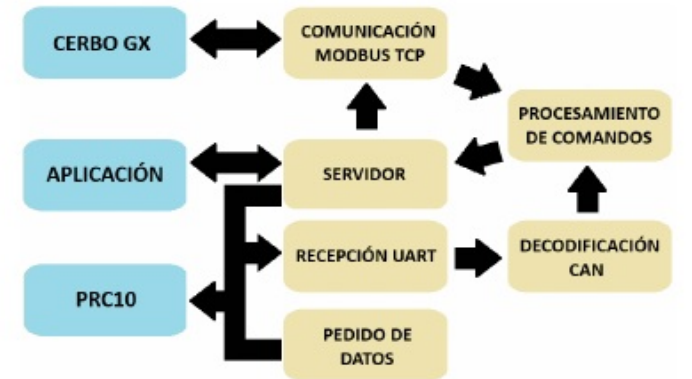


FIGURA 3.6. Relación entre las distintas tareas del firmware y su comunicación con el exterior.

3.2.1. Tarea: recepción UART

Esta tarea se encarga de monitorear de manera continua el puerto UART del ESP32. Cada vez que se detecta una señal en sus puertos, la tarea almacena la información recibida en un buffer de tipo circular [34].

Un buffer circular, también conocido como *ring buffer*, es una estructura de datos que utiliza una región de memoria de tamaño fijo y organiza los datos de forma continua. Cuando se alcanza el final del buffer, la escritura continúa desde el inicio, lo que permite reutilizar la memoria de manera eficiente sin necesidad de realizar desplazamientos de datos.

Se optó por este tipo de estructura porque resulta adecuada para la recepción de datos en tiempo real, ya que permite desacoplar la velocidad de llegada de la información respecto de su procesamiento. De esta forma, se evita la pérdida de datos ante ráfagas de información y se garantiza un manejo más robusto de la comunicación UART.

3.2.2. Tarea: decodificación de comando CAN

Esta tarea se encarga de decodificar los comandos recibidos desde la red CAN. Para ello, realiza la lectura de los datos almacenados en el buffer circular.

En cuanto al módulo W5100, se optó por utilizar la versión comercial disponible en el mercado. Como consecuencia, en el PCB se reservó un espacio para la colocación de un conector compatible con el módulo W5100, lo que permite su montaje manual de forma sencilla.

En la figura 3.6 se puede observar el diseño final del PCB.

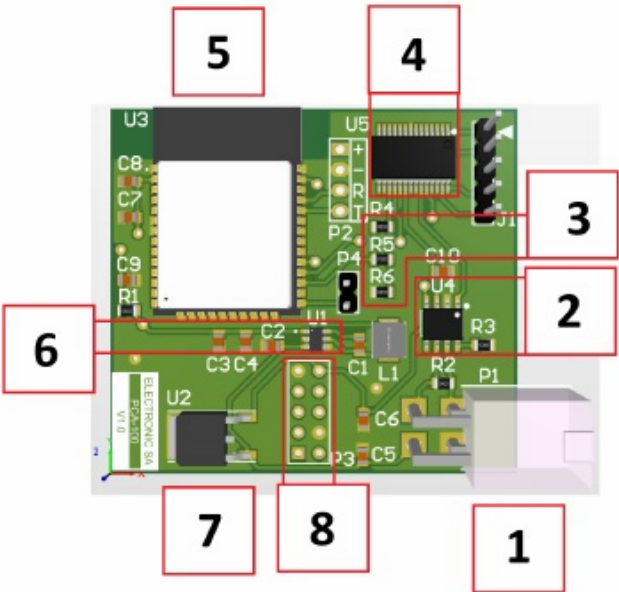


FIGURA 3.6. Diseño final para PCB del equipo.

Como se puede apreciar, el PCB está compuesto por los siguientes componentes:

- Componente 1: conector principal. Terminal sobre el que se conecta el cableado de cuatro hilos proveniente del motorhome.
- Componente 2: MCP2551. Forma parte del módulo transceptor, encargado de adaptar los niveles de tensión de la línea CAN.
- Componente 3: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceptor.
- Componente 4: PIC18F26K80. Microcontrolador que contiene la biblioteca desarrollada por Enertik para la comunicación entre equipos.
- Componente 5: módulo ESP32. Microcontrolador que contiene el firmware principal del equipo.
- Componente 6: regulador de tensión AP63203. Fuente de tensión de 3,3 V encargado de alimentar el ESP32.

La información transmitida en la red CAN sigue un formato predefinido. El primer byte corresponde a una marca de inicio, que permite identificar el comienzo de cada trama. Por otro lado, el último byte corresponde a un valor de control CRC (Cyclic Redundancy Check), utilizado para verificar la integridad de los datos recibidos mediante un cálculo basado en el contenido del mensaje.

La trama incluye un campo de dirección de origen para identificar al dispositivo emisor, un campo de dirección de destino para indicar el dispositivo receptor, un campo de comando asociado a la acción a ejecutar y un conjunto de bytes de datos.

El análisis de cada trama completa se realiza mediante el recorrido del buffer circular hasta encontrar el byte de marca de inicio.

A continuación, extrae la trama completa y realiza el cálculo del CRC, lo que permite determinar si la trama es válida o no. En caso afirmativo, se procede a identificar los campos que componen el mensaje.

Cuando se detecta una trama válida, la información útil se transfiere a la siguiente tarea, encargada de procesar la instrucción correspondiente.

3.2.3. Tarea: comunicación Modbus TCP

Esta tarea se encarga de gestionar la comunicación Modbus TCP y, por lo tanto, es responsable del control del módulo Cerbo GX.

Para establecer la comunicación mediante este protocolo, fue necesario realizar una serie de configuraciones iniciales.

Se definió que el ESP32 ejerza el rol de maestro, encargado de solicitar datos y enviar comandos al Cerbo GX.

La comunicación entre ambos dispositivos se realiza a través de un cable Ethernet con conector RJ45, que vincula el Cerbo GX con el módulo W5100 controlado por el ESP32.

Para que esta comunicación sea posible, fue necesario definir direcciones IP estáticas para ambos dispositivos. Esto permite que cada equipo conozca la dirección del otro y mantenga una comunicación consistente en el tiempo. Además, ambos deben pertenecer a la misma red, lo que implica compartir la misma máscara de subred y una configuración de puerta de enlace (gateway) compatible.

La dirección IP del Cerbo GX se configura desde el menú del propio equipo. Por otro lado, la dirección IP del ESP32 se definió por software dentro de esta tarea.

Para su funcionamiento, se utiliza un contador interno que cumple la función de temporizador. En primer lugar, establece la comunicación Modbus TCP y se la mantiene activa durante el funcionamiento del sistema.

Posteriormente, en intervalos periódicos, realiza la lectura de los registros que el Cerbo GX pone a disposición para la obtención de datos y el control del comportamiento del equipo.

En este contexto, los parámetros de interés fueron aquellos asociados al modo de operación del equipo, ya sea en modo inversor o cargador.

- Componente 7: regulador de tensión 7805. Componente responsable de suministrar una tensión estable de 5 V para alimentar el PIC y el W5100.
- Componente 8: conector de diez pines. Conector compatible con el módulo W5100. En este lugar se montará dicho componente.

Para el proceso de fabricación, fue necesaria la contratación de servicios de un fabricante especializado en este campo. Para ello, se generaron los denominados archivos Gerber [33].

A modo de resumen, la figura 3.7 muestra todos los componentes de hardware empleados en el diseño y como se relacionan unos con otros.

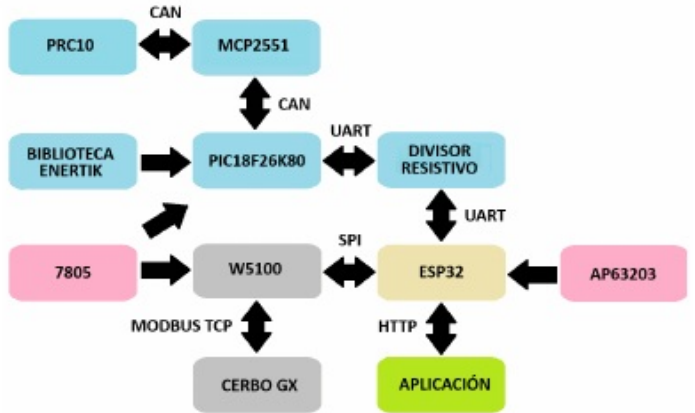


FIGURA 3.7. Diagrama final de los componentes de hardware empleados.

3.2. Desarrollo de firmware

Como se mencionó en la sección 2.4.2, se optó por utilizar FreeRTOS como base para el desarrollo del firmware.

FreeRTOS permite estructurar el firmware mediante la ejecución de múltiples tareas. Su planificación genera un comportamiento concurrente a nivel lógico, lo que mejora la organización del sistema y facilita la gestión de sus funcionalidades.

En este contexto, para el presente trabajo se implementaron seis tareas principales, ilustradas en la figura 3.8, donde se aprecian las relaciones existentes entre ellas y con los módulos externos.

Victron dispone de un registro que adopta distintos valores según el estado del equipo: modo inversor, modo cargador, funcionamiento combinado o estado apagado. A partir de la lectura de este registro es posible determinar el estado del equipo en todo momento.

Además, este registro admite operaciones de lectura y escritura, lo que permite modificar el comportamiento del equipo mediante la asignación de valores adecuados.

Otros parámetros relevantes incluyen la potencia suministrada por el equipo en modo inversor y la corriente de carga de entrada en modo cargador.

3.2.4. Tarea: pedido de datos

Esta tarea se encarga de solicitar de forma periódica información al módulo PRC10.

Dentro de la red de Enertik, el módulo PRC10 actúa como unidad central de procesamiento. En este módulo se concentran los valores y estados de todas las variables del motorhome.

Para que la aplicación pueda reflejar esta información en pantalla, resulta necesario que el ESP32 tenga acceso a dichos datos.

Con este objetivo, se realizó una modificación en el firmware del módulo PRC10. Se incorporó una función que permite generar una serie de paquetes, cada uno asociado a un conjunto de variables del sistema.

Entre las variables incluidas en estos paquetes se encuentran:

- Tensión de batería.
- Corriente suministrada por la batería.
- Tensión de entrada en alterna.
- Frecuencia de la tensión de entrada.
- Origen de la tensión de entrada (red domiciliaria, generador o inversor).
- Nivel de llenado de los tanques de agua (limpia, gris y negra).
- Estado del cargador.
- Corriente de carga.
- Estado del inversor.
- Potencia suministrada por el inversor.
- Estado de los pulsadores asociados a las distintas pantallas de la HMI.

Desde el lado del ESP32, esta tarea realiza solicitudes periódicas de cada uno de estos paquetes. El funcionamiento se basa en un ciclo en el que, cada 500 ms, se solicita un paquete distinto al PRC10.

Una vez completado el recorrido de todos los paquetes, el proceso se reinicia desde el primero. Este comportamiento cíclico permite mantener actualizada la información correspondiente a todas las variables del sistema.

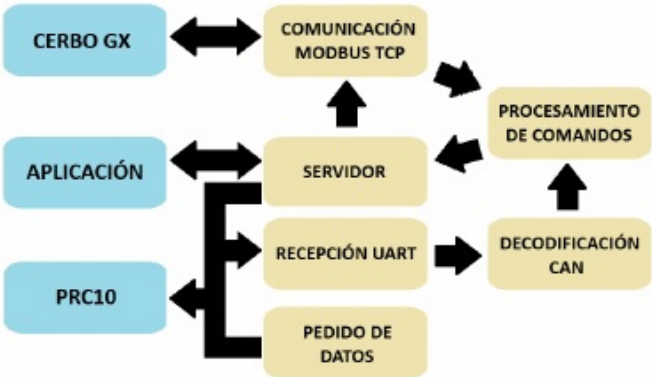


FIGURA 3.8. Relación entre las distintas tareas del firmware y su comunicación con el exterior.

3.2.1. Tarea: recepción UART

Esta tarea se encarga de monitorear de manera continua el puerto UART del ESP32. Cada vez que se detecta una señal en sus puertos, la tarea almacena la información recibida en un buffer de tipo circular [34].

Un buffer circular, también conocido como *ring buffer*, es una estructura de datos que utiliza una región de memoria de tamaño fijo y organiza los datos de forma continua. Cuando se alcanza el final del buffer, la escritura continúa desde el inicio, lo que permite reutilizar la memoria de manera eficiente sin necesidad de realizar desplazamientos de datos.

Se optó por este tipo de estructura porque resulta adecuada para la recepción de datos en tiempo real, ya que permite desacoplar la velocidad de llegada de la información respecto de su procesamiento. De esta forma, se evita la pérdida de datos ante ráfagas de información y se garantiza un manejo más robusto de la comunicación UART.

3.2.2. Tarea: decodificación de comando CAN

Esta tarea se encarga de decodificar los comandos recibidos desde la red CAN. Para ello, realiza la lectura de los datos almacenados en el buffer circular.

La información transmitida en la red CAN sigue un formato predefinido. El primer byte corresponde a una marca de inicio, que permite identificar el comienzo de cada trama. Por otro lado, el último byte corresponde a un valor de control CRC (*Cyclic Redundancy Check*), utilizado para verificar la integridad de los datos recibidos mediante un cálculo basado en el contenido del mensaje.

La trama incluye un campo de dirección de origen para identificar al dispositivo emisor, un campo de dirección de destino para indicar el dispositivo receptor, un campo de comando asociado a la acción a ejecutar y un conjunto de bytes de datos.

3.2.5. Tarea: procesamiento de comandos recibidos

Esta tarea constituye el núcleo del firmware. Su función principal es recibir los comandos provenientes de las tareas de comunicación CAN (3.2.2) y Modbus TCP (3.2.3), procesarlos y actualizar las variables internas del sistema.

En relación con la comunicación CAN, esta tarea recibe las tramas ya validadas por la etapa de decodificación. A partir de estas, extrae los distintos campos de datos y los interpreta según el tipo de paquete recibido. Los valores obtenidos se almacenan en estructuras de datos internas, que representan el estado actualizado de los distintos sensores y actuadores del motorhome.

Adicionalmente, se realizan conversiones sobre los datos recibidos, ya que muchos de ellos se encuentran codificados. Estas conversiones permiten obtener magnitudes físicas en unidades reales, lo que facilita su posterior visualización y procesamiento.

Por otra parte, la tarea también gestiona comandos destinados al módulo Cerbo GX a través de Modbus TCP. En función del comando recibido, se determinan las acciones a ejecutar, como la activación o desactivación del inversor o del cargador. Para ello, se realiza la escritura sobre los registros correspondientes del dispositivo Victron, lo que permite modificar su estado de operación.

Finalmente, una vez procesada la información, los datos actualizados se envían a otras tareas del sistema a través de colas de FreeRTOS. Este mecanismo permite desacoplar el procesamiento de datos de su consumo, lo que mejora la organización y la escalabilidad del firmware.

3.2.6. Tarea: control de servidor

Esta tarea gestiona la implementación del servidor en el ESP32.

En primer lugar, se encarga de configurar el dispositivo en modo *access point*. En esta modalidad, el microcontrolador genera su propia red Wi-Fi, lo que permite la conexión directa de otros dispositivos. Además, se definen los parámetros de la red, tales como el nombre (SSID) y la contraseña de acceso.

Una vez establecida la conectividad, se definió la estructura del servidor HTTP mediante la creación de distintos endpoints. Estos representan puntos de acceso que permiten a los clientes interactuar con el sistema. Cada endpoint se asocia a una ruta específica y a un tipo de solicitud, como GET o POST, lo que permite consultar información o enviar comandos.

Se optó por implementar un endpoint para cada una de las pantallas principales de la aplicación. Este enfoque permite que cada pantalla solicite únicamente la información necesaria, lo que reduce el tráfico de datos y mejora la eficiencia del sistema.

En particular, se definieron cuatro endpoints que admiten solicitudes de tipo GET: home, luces, motores y energía.

Cada uno de estos endpoints genera y devuelve un archivo en formato JSON (*JavaScript Object Notation*) [35] con la información requerida por la pantalla correspondiente, lo que permite actualizar la interfaz de usuario con los valores actuales del sistema.

El análisis de cada trama completa se realiza mediante el recorrido del buffer circular hasta encontrar el byte de marca de inicio.

A continuación, extrae la trama completa y realiza el cálculo del CRC, lo que permite determinar si la trama es válida o no. En caso afirmativo, se procede a identificar los campos que componen el mensaje.

Cuando se detecta una trama válida, la información útil se transfiere a la siguiente tarea, encargada de procesar la instrucción correspondiente.

3.2.3. Tarea: comunicación Modbus TCP

Esta tarea se encarga de gestionar la comunicación Modbus TCP y, por lo tanto, es responsable del control del módulo Cerbo GX.

Para establecer la comunicación mediante este protocolo, fue necesario realizar una serie de configuraciones iniciales.

Se definió que el ESP32 ejerza el rol de maestro, encargado de solicitar datos y enviar comandos al Cerbo GX.

La comunicación entre ambos dispositivos se realiza a través de un cable Ethernet con conector RJ45, que vincula el Cerbo GX con el módulo W5100 controlado por el ESP32.

Para que esta comunicación sea posible, fue necesario definir direcciones IP estáticas para ambos dispositivos. Esto permite que cada equipo conozca la dirección del otro y mantenga una comunicación consistente en el tiempo. Además, ambos deben pertenecer a la misma red, lo que implica compartir la misma máscara de subred y una configuración de puerta de enlace (gateway) compatible.

La dirección IP del Cerbo GX se configura desde el menú del propio equipo. Por otro lado, la dirección IP del ESP32 se definió por software dentro de esta tarea.

Para su funcionamiento, se utiliza un contador interno que cumple la función de temporizador. En primer lugar, establece la comunicación Modbus TCP y se la mantiene activa durante el funcionamiento del sistema.

Posteriormente, en intervalos periódicos, realiza la lectura de los registros que el Cerbo GX pone a disposición para la obtención de datos y el control del comportamiento del equipo.

En este contexto, los parámetros de interés fueron aquellos asociados al modo de operación del equipo, ya sea en modo inversor o cargador.

Victron dispone de un registro que adopta distintos valores según el estado del equipo: modo inversor, modo cargador, funcionamiento combinado o estado apagado. A partir de la lectura de este registro es posible determinar el estado del equipo en todo momento.

Además, este registro admite operaciones de lectura y escritura, lo que permite modificar el comportamiento del equipo mediante la asignación de valores adecuados.

Otros parámetros relevantes incluyen la potencia suministrada por el equipo en modo inversor y la corriente de carga de entrada en modo cargador.

Por otra parte, la aplicación móvil dispone de distintos pulsadores que permiten controlar el funcionamiento del motorhome. Para ello, se implementó un quinto endpoint (denominado pulsadores) que admite solicitudes de tipo POST, destinado a la recepción de comandos.

Este endpoint recibe un archivo JSON que contiene dos campos: uno que identifica el pulsador que originó la acción y otro que indica el nuevo estado asociado.

A partir de esta información, el servidor interpreta el comando y lo traduce en una instrucción que es enviada al módulo PRC10 para así permitir la ejecución de la acción correspondiente dentro del sistema.

De esta manera, el servidor implementado en el ESP32 actúa como intermediario entre la aplicación móvil y la red de domótica, lo que permite centralizar tanto la consulta de información como el envío de comandos.

3.3. Desarrollo de la aplicación

Para el desarrollo de la aplicación se optó por utilizar el framework Ionic. Su elección aportó diversas ventajas durante el proceso de desarrollo.

En primer lugar, permitió utilizar el navegador web de la computadora como herramienta de depuración, lo que facilitó la visualización y prueba de la aplicación en tiempo real. Esto redujo significativamente los tiempos de desarrollo, ya que cada modificación realizada podía verificarse de forma inmediata.

Asimismo, este entorno facilitó la realización de ensayos sobre los endpoints definidos en el servidor del ESP32. Para ello, el dispositivo se conectó a la misma red Wi-Fi que la computadora de desarrollo, lo que permitió que la aplicación realizara solicitudes HTTP de tipo GET y POST hacia los endpoints correspondientes.

Estas solicitudes se dirigieron a la dirección IP asignada al ESP32, dato que el propio dispositivo informa al momento de establecer la conexión a la red.

Para el desarrollo de la aplicación, se tomó como referencia el software utilizado en la pantalla HMI del motorhome. Se replicaron la mayor cantidad posible de sus funcionalidades, con prioridad en aquellas consideradas de mayor importancia.

Como se mencionó en la sección 3.2.6, se optó por desarrollar una aplicación compuesta por cuatro pestañas. A su vez, se implementó una barra de navegación por pestañas que permite acceder a las distintas pantallas de la aplicación.

Previo a la implementación, se definió un criterio de diseño orientado a la organización modular de la interfaz. Para ello, se adoptó un enfoque basado en contenedores visuales personalizados, cuyo tamaño se define en términos relativos al espacio disponible en pantalla.

La figura 3.7 ilustra este criterio. Cada contenedor puede ocupar un porcentaje del ancho y del alto de la pantalla, lo que permite estructurar la interfaz mediante una disposición en filas y columnas.

A partir de este esquema, las pantallas se construyen como una composición de contenedores anidados. Dentro cada bloque final se integra cada uno de los elementos funcionales de la aplicación, tales como indicadores, controles y visualizaciones de datos.

3.2.4. Tarea: pedido de datos

Esta tarea se encarga de solicitar de forma periódica información al módulo PRC10.

Dentro de la red de Enertik, el módulo PRC10 actúa como unidad central de procesamiento. En este módulo se concentran los valores y estados de todas las variables del motorhome.

Para que la aplicación pueda reflejar esta información en pantalla, resulta necesario que el ESP32 tenga acceso a dichos datos.

Con este objetivo, se realizó una modificación en el firmware del módulo PRC10. Se incorporó una función que permite generar una serie de paquetes, cada uno asociado a un conjunto de variables del sistema.

Entre las variables incluidas en estos paquetes se encuentran:

- Tensión de batería.
- Corriente suministrada por la batería.
- Tensión de entrada en alterna.
- Frecuencia de la tensión de entrada.
- Origen de la tensión de entrada (red domiciliaria, generador o inversor).
- Nivel de llenado de los tanques de agua (limpia, gris y negra).
- Estado del cargador.
- Corriente de carga.
- Estado del inversor.
- Potencia suministrada por el inversor.
- Estado de los pulsadores asociados a las distintas pantallas de la HMI.

Desde el lado del ESP32, esta tarea realiza solicitudes periódicas de cada uno de estos paquetes. El funcionamiento se basa en un ciclo en el que, cada 500 ms, se solicita un paquete distinto al PRC10.

Una vez completado el recorrido de todos los paquetes, el proceso se reinicia desde el primero. Este comportamiento cíclico permite mantener actualizada la información correspondiente a todas las variables del sistema.

3.2.5. Tarea: procesamiento de comandos recibidos

Esta tarea constituye el núcleo del firmware. Su función principal es recibir los comandos provenientes de las tareas de comunicación CAN (3.2.2) y Modbus TCP (3.2.3), procesarlos y actualizar las variables internas del sistema.

En relación con la comunicación CAN, esta tarea recibe las tramas ya validadas por la etapa de decodificación. A partir de estas, extrae los distintos campos de datos y los interpreta según el tipo de paquete recibido. Los valores obtenidos se almacenan en estructuras de datos internas, que representan el estado actualizado de los distintos sensores y actuadores del motorhome.



FIGURA 3.7. Contenedores dentro de una pantalla.

Este enfoque responde a un modelo de layout basado en grillas, ampliamente utilizado en el desarrollo de interfaces modernas. Su uso permitió mantener una distribución consistente de los elementos en todas las pantallas y facilitó la adaptación del diseño a distintos tamaños de dispositivo.

En el ejemplo de la figura 3.7, se observa un contenedor principal que define el área de trabajo de la pantalla. A partir de este, se establecen subdivisiones en filas y columnas mediante nuevos contenedores, lo que permite organizar la ubicación de cada elemento de la interfaz.

Inicialmente, estos contenedores se representaron con colores para facilitar la visualización de la estructura. Una vez definida la disposición final, se ajustó su apariencia según el diseño de cada pantalla y, en la mayoría de los casos, se utilizó un fondo transparente para integrarlos visualmente con la interfaz.

3.3.1. Arquitectura funcional de la aplicación

La aplicación móvil se diseñó con una estructura basada en navegación por pestañas, con el objetivo de organizar las distintas funciones del sistema en módulos independientes.

Como se mencionó en la sección 3.2.6, se definieron cuatro vistas principales: home, luces, motores y energía. Cada una de ellas agrupa funcionalidades asociadas a un subsistema específico del motorhome, lo que simplifica la navegación y mejora la experiencia de uso.

La figura 3.8 presenta el mapa de navegación de la aplicación. En ella se observan las distintas vistas definidas y los principales elementos funcionales asociados a cada una de ellas, entre los que se incluyen variables de monitoreo, indicadores y controles.

Adicionalmente, se realizan conversiones sobre los datos recibidos, ya que muchos de ellos se encuentran codificados. Estas conversiones permiten obtener magnitudes físicas en unidades reales, lo que facilita su posterior visualización y procesamiento.

Por otra parte, la tarea también gestiona comandos destinados al módulo Cerbo GX a través de Modbus TCP. En función del comando recibido, se determinan las acciones a ejecutar, como la activación o desactivación del inversor o del cargador. Para ello, se realiza la escritura sobre los registros correspondientes del dispositivo Victron, lo que permite modificar su estado de operación.

Finalmente, una vez procesada la información, los datos actualizados se envían a otras tareas del sistema a través de colas de FreeRTOS. Este mecanismo permite desacoplar el procesamiento de datos de su consumo, lo que mejora la organización y la escalabilidad del firmware.

3.2.6. Tarea: control de servidor

Esta tarea gestiona la implementación del servidor en el ESP32.

En primer lugar, se encarga de configurar el dispositivo en modo *Access Point*. En esta modalidad, el microcontrolador genera su propia red Wi-Fi, lo que permite la conexión directa de otros dispositivos. Además, se definen los parámetros de la red, tales como el nombre (SSID) y la contraseña de acceso.

Una vez establecida la conectividad, se definió la estructura del servidor HTTP mediante la creación de distintos endpoints. Estos representan puntos de acceso que permiten a los clientes interactuar con el sistema. Cada endpoint se asocia a una ruta específica y a un tipo de solicitud, como GET o POST, lo que permite consultar información o enviar comandos.

Se optó por implementar un endpoint para cada una de las pantallas principales de la aplicación. Este enfoque permite que cada pantalla solicite únicamente la información necesaria, lo que reduce el tráfico de datos y mejora la eficiencia del sistema.

En particular, se definieron cuatro endpoints que admiten solicitudes de tipo GET:

- Home
- Luces
- Motores
- Energía

Cada uno de estos endpoints genera y devuelve un archivo en formato *JSON* (*JavaScript Object Notation*) [35] con la información requerida por la pantalla correspondiente, lo que permite actualizar la interfaz de usuario con los valores actuales del sistema.

Por otra parte, la aplicación móvil dispone de distintos pulsadores que permiten controlar el funcionamiento del motorhome. Para ello, se implementó un quinto endpoint (denominado pulsadores) que admite solicitudes de tipo POST, destinado a la recepción de comandos.

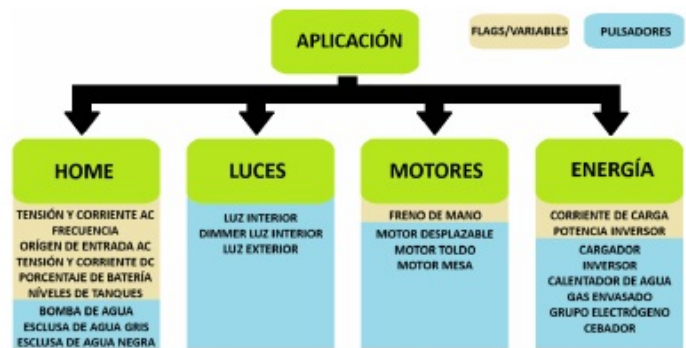


FIGURA 3.8. Arquitectura funcional de la aplicación.

La vista home constituye la pantalla principal de la aplicación y centraliza las variables más relevantes del sistema. En ella se incluyen magnitudes asociadas al sistema de baterías, variables de corriente alterna, niveles de los tanques de agua y controles correspondientes a actuadores de uso frecuente, como la bomba de agua y las esclusas.

La vista luces concentra las funciones asociadas al sistema de iluminación del motorhome. Permite supervisar el estado de los circuitos de iluminación interior y exterior, así como regular la intensidad de la iluminación interior mediante un control de tipo dimmer.

La vista motores agrupa el control de los elementos móviles del motorhome, tales como la mesa, el toldo y la cabina desplazable. Además, incorpora restricciones lógicas de seguridad vinculadas al estado del freno de mano del vehículo.

Finalmente, la vista energía centraliza el monitoreo y control de los principales subsistemas energéticos, entre ellos el grupo electrógeno, el cargador, el inversor y otros consumos de alta potencia vinculados al sistema.

3.3.2. Secuencia de interacción entre la aplicación y el servidor

El funcionamiento de la aplicación se basa en una arquitectura cliente-servidor, en la que el dispositivo móvil actúa como cliente y el ESP32 como servidor encargado de procesar las solicitudes y administrar el acceso a las variables del sistema.

La comunicación entre ambos dispositivos se implementó mediante el protocolo HTTP. En este esquema, la actualización de variables se realiza a través de solicitudes de tipo GET, mientras que las acciones de control se ejecutan mediante solicitudes de tipo POST.

La figura 3.9 representa la secuencia de intercambio correspondiente a la actualización periódica de variables.

Cada vez que el usuario accede a una pestaña de la aplicación, se activa un temporizador interno configurado con un período de tres segundos.

Este endpoint recibe un archivo JSON que contiene dos campos: uno que identifica el pulsador que originó la acción y otro que indica el nuevo estado asociado.

A partir de esta información, el servidor interpreta el comando y lo traduce en una instrucción que es enviada al módulo PRC10 para así permitir la ejecución de la acción correspondiente dentro del sistema.

De esta manera, el servidor implementado en el ESP32 actúa como intermediario entre la aplicación móvil y la red de domótica, lo que permite centralizar tanto la consulta de información como el envío de comandos.

3.3. Desarrollo de la aplicación

Para el desarrollo de la aplicación se optó por utilizar el framework Ionic. Su elección aportó diversas ventajas durante el proceso de desarrollo.

En primer lugar, permitió utilizar el navegador web de la computadora como herramienta de depuración, lo que facilitó la visualización y prueba de la aplicación en tiempo real. Esto redujo significativamente los tiempos de desarrollo, ya que cada modificación realizada podía verificarse de forma inmediata.

Asimismo, este entorno facilitó la realización de ensayos sobre los endpoints definidos en el servidor del ESP32. Para ello, el dispositivo se conectó a la misma red Wi-Fi que la computadora de desarrollo, lo que permitió que la aplicación realizara solicitudes HTTP de tipo GET y POST hacia los endpoints correspondientes.

Estas solicitudes se dirigieron a la dirección IP asignada al ESP32, dato que el propio dispositivo informa al momento de establecer la conexión a la red.

Para el desarrollo de la aplicación, se tomó como referencia el software utilizado en la pantalla HMI del motorhome. Se replicaron la mayor cantidad posible de sus funcionalidades, con prioridad en aquellas consideradas de mayor importancia.

Como se mencionó en la sección 3.2.6, se optó por desarrollar una aplicación compuesta por cuatro pestañas. A su vez, se implementó una barra de navegación por pestañas que permite acceder a las distintas pantallas de la aplicación.

Previo a la implementación, se definió un criterio de diseño orientado a la organización modular de la interfaz. Para ello, se adoptó un enfoque basado en contenedores visuales personalizados, cuyo tamaño se define en términos relativos al espacio disponible en pantalla.

La figura 3.9 ilustra este criterio. Cada contenedor puede ocupar un porcentaje del ancho y del alto de la pantalla, lo que permite estructurar la interfaz mediante una disposición en filas y columnas.

A partir de este esquema, las pantallas se construyen como una composición de contenedores anidados. Dentro cada bloque final se integra cada uno de los elementos funcionales de la aplicación, tales como indicadores, controles y visualizaciones de datos.

Este enfoque responde a un modelo de layout basado en grillas, ampliamente utilizado en el desarrollo de interfaces modernas. Su uso permitió mantener una distribución consistente de los elementos en todas las pantallas y facilitó la adaptación del diseño a distintos tamaños de dispositivo.

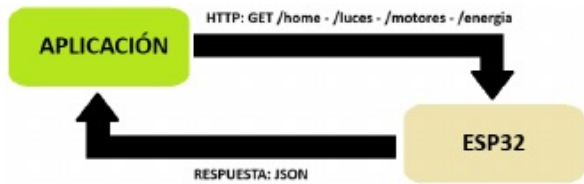


FIGURA 3.9. Secuencia de solicitudes GET entre la aplicación y el microcontrolador.

Al finalizar cada período, la pestaña activa envía una solicitud GET a su endpoint asociado. El ESP32 procesa dicha solicitud y responde con un archivo en formato `JSON` que contiene las variables correspondientes a esa vista, junto con el estado actual de sus controles.

Este mecanismo permite mantener sincronizada la información mostrada en pantalla con el estado real del sistema.

Por otra parte, las acciones de control se realizan mediante solicitudes HTTP de tipo POST. La figura 3.10 representa esta secuencia de comunicación.

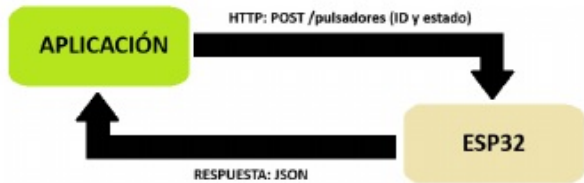


FIGURA 3.10. Secuencia de solicitudes POST entre la aplicación y el microcontrolador.

Cada vez que el usuario acciona un pulsador desde la interfaz, la aplicación genera una solicitud POST dirigida al endpoint pulsadores. En esta solicitud se transmite la identificación del control accionado y el nuevo estado requerido.

Luego, el ESP32 procesa la petición, ejecuta la lógica de control correspondiente y envía una respuesta de confirmación en formato `JSON`. Esta respuesta informa el nuevo estado del elemento controlado y permite mantener la coherencia entre el sistema físico y la interfaz de usuario.

3.3.3. Generación de instalador para Android

Una vez finalizado el desarrollo de la aplicación, fue necesario generar un instalador que permita su ejecución en dispositivos móviles con sistema operativo Android. Para ello, se utilizó la herramienta Capacitor, que posibilita la adaptación de aplicaciones desarrolladas con tecnologías web a un entorno nativo.

Capacitor actúa como una capa de integración entre la aplicación desarrollada con Ionic y el sistema operativo del dispositivo. A través de esta herramienta, la aplicación se empaqueta en una estructura compatible con Android. Además, brinda acceso a funcionalidades propias del dispositivo.



FIGURA 3.9. Contenedores dentro de una pantalla.

En el ejemplo de la figura 3.9, se observa un contenedor principal que define el área de trabajo de la pantalla. A partir de este, se establecen subdivisiones en filas y columnas mediante nuevos contenedores, lo que permite organizar la ubicación de cada elemento de la interfaz.

Inicialmente, estos contenedores se representaron con colores para facilitar la visualización de la estructura. Una vez definida la disposición final, se ajustó su apariencia según el diseño de cada pantalla y, en la mayoría de los casos, se utilizó un fondo transparente para integrarlos visualmente con la interfaz.

3.3.1. Aplicación: pestaña principal

La pestaña principal, representada en la figura 3.10, constituye el punto de acceso central a la información del sistema, ya que concentra las variables más relevantes asociadas al estado general del motorhome.



FIGURA 3.10. Pestaña principal de la aplicación.

En este contexto, se emplearon herramientas provistas por Capacitor para la gestión de solicitudes HTTP desde la aplicación, y permitir así un control más directo sobre la comunicación con el servidor del ESP32. Asimismo, se utilizaron utilidades para la generación y configuración de los recursos gráficos de la aplicación, como los íconos y elementos visuales requeridos por el sistema Android.

A partir de este proceso, el trabajo fue importado en el entorno de desarrollo Android Studio. Este software permite gestionar la compilación de la aplicación, así como configurar parámetros propios del sistema Android, tales como permisos, versión mínima del sistema operativo y opciones de despliegue.

Como resultado de la compilación, se obtiene un archivo en formato APK, que constituye el formato estándar de distribución de aplicaciones para el sistema operativo Android. Este archivo contiene todos los recursos necesarios para la instalación de la aplicación, tales como el código ejecutable, las bibliotecas y los archivos de configuración.

La instalación del APK puede realizarse directamente sobre un dispositivo móvil mediante distintas alternativas. En este trabajo se utilizó la herramienta ADB (*Android Debug Bridge*) [36], que permite la transferencia e instalación del archivo desde una computadora hacia el dispositivo a través de una conexión USB. Una vez instalado, el usuario puede ejecutar la aplicación de forma independiente, sin necesidad de herramientas de desarrollo adicionales.

Desde el punto de vista funcional, esta pantalla implementa un mecanismo de actualización periódica basado en solicitudes HTTP de tipo GET al endpoint home.

Para ello, se utiliza un temporizador interno que permite consultar de forma cíclica el estado del sistema. De este modo, la información visualizada se mantiene sincronizada con las variables reales del motorhome.

La información presentada se organiza en distintos grupos funcionales. Por un lado, se incluyen variables asociadas al sistema de baterías, tales como tensión, corriente y porcentaje de carga. Por otro lado, se presentan magnitudes correspondientes al sistema de corriente alterna, entre las que se encuentran la tensión, la corriente y la frecuencia de entrada.

Adicionalmente, se incorporan indicadores que permiten identificar el origen de la alimentación del sistema. Esta lógica responde al comportamiento del módulo PRC10, que establece un esquema de prioridad entre las distintas fuentes disponibles.

En este esquema, la línea de alimentación posee la mayor prioridad, seguida por el inversor y, finalmente, el grupo electrógeno. La interfaz refleja este criterio de forma visual, lo que permite interpretar rápidamente el estado de operación del sistema.

Asimismo, la pantalla incluye la visualización del nivel de los tanques de agua, representados en términos porcentuales, lo que permite un monitoreo directo de estos recursos.

En cuanto al control del sistema, se incorporan pulsadores que permiten accionar distintos actuadores del motorhome, como la bomba de agua y las esclusas de los tanques. Estos comandos se envían mediante solicitudes HTTP de tipo POST al endpoint pulsadores, donde se transmite la identificación del control y su nuevo estado en formato JSON.

De esta manera, la pestaña principal integra en una única interfaz la visualización de variables críticas y el acceso a funciones de control, y representa el principal punto de interacción entre el usuario y el sistema.

3.3.2. Aplicación: pestaña luces

La pestaña correspondiente al control de iluminación, representada en la figura 3.11, concentra las funciones asociadas al manejo de las luces del sistema.

Desde el punto de vista funcional, esta pantalla implementa un mecanismo de actualización periódica mediante solicitudes HTTP de tipo GET al endpoint luces. A través de estas consultas, se obtienen los estados de los circuitos de iluminación y el valor asociado al control de intensidad.

En particular, el sistema permite gestionar tanto la iluminación interior como la exterior. Para el caso de la iluminación interior, se dispone de un control de intensidad basado en modulación por ancho de pulso, implementado en el módulo PRC10. Esto permite ajustar el nivel de iluminación mediante la variación del ciclo de trabajo de la señal.

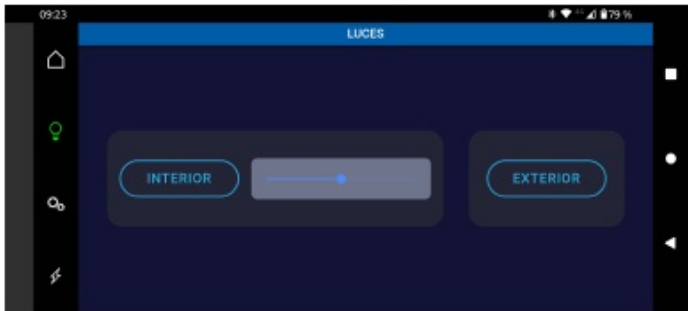


FIGURA 3.11. Pestaña luces de la aplicación.

Las acciones de control se envían mediante solicitudes HTTP de tipo POST al endpoint pulsadores, donde se transmite la identificación del elemento a accionar y el valor correspondiente. Este esquema unifica el mecanismo de control para distintos tipos de actuadores, tanto discretos como analógicos.

De esta manera, la pestaña de iluminación permite supervisar el estado de los circuitos y ajustar su comportamiento a través de una interfaz simple, en coherencia con la arquitectura de comunicación definida para el sistema.

3.3.3. Aplicación: pestaña motores

La pestaña de motores, representada en la figura 3.12, se encarga de la supervisión y el control de los elementos móviles del motorhome.

Desde el punto de vista funcional, esta pantalla implementa un mecanismo de actualización periódica mediante solicitudes HTTP de tipo GET al endpoint motores. A través de estas consultas, se obtienen tanto los estados de los actuadores como las variables necesarias para la lógica de control.

La interfaz permite seleccionar el elemento móvil sobre el que se desea operar y presentar los controles asociados a dicho elemento. De esta forma, se reutiliza una misma estructura de control para distintos actuadores, lo que simplifica el diseño de la aplicación.

Entre las variables recibidas se incluye el estado del freno de mano del vehículo. Esta señal se utiliza como condición de habilitación para determinados actuadores, en particular la cabina desplazable y el toldo.

Cuando el freno de mano no se encuentra accionado, el sistema bloquea la interacción con estos controles. Esta restricción responde a criterios de seguridad, ya que dichos mecanismos solo deben operar con el vehículo detenido. Una vez que el sistema detecta la activación del freno de mano, los controles correspondientes se habilitan.

Por otro lado, el motor asociado al desplazamiento de la mesa no depende de esta condición, por lo que su control permanece habilitado en todo momento.

Capítulo 4

Ensayos y resultados

En este capítulo se explica cómo fueron los ensayos que se realizaron sobre el sistema y qué resultados se obtuvieron a partir de ellos.

4.1. Diseño final de la aplicación

Como resultado del proceso de desarrollo descrito en la sección 3.3, se obtuvo una aplicación móvil funcional orientada al monitoreo y control de distintos subsistemas del motorhome.

La interfaz final quedó organizada en cuatro pestañas principales: home, luces, motores y energía. Cada una de ellas fue diseñada para centralizar funciones específicas del sistema y facilitar la interacción del usuario con los distintos módulos controlados por el ESP32.

La figura 4.1 presenta las cuatro pantallas finales de la aplicación desarrollada.



FIGURA 4.1. Diseño final de las principales pantallas de la aplicación móvil.

Estas pantallas constituyen la versión final utilizada durante los ensayos presentados en las secciones siguientes.



FIGURA 3.12. Pestaña motores de la aplicación.

Las acciones de control se envían mediante solicitudes HTTP de tipo POST al endpoint pulsadores, donde se transmite la identificación del actuador y el estado requerido.

De esta manera, la pestaña de motores centraliza el control de los elementos móviles e incorpora restricciones de operación que garantizan un uso seguro del sistema.

3.3.4. Aplicación: pestaña energía

La pestaña de energía, representada en la figura 3.13, concentra la supervisión y el control de los principales sistemas energéticos del motorhome, entre ellos el grupo electrógeno, el cargador, el inversor y otros consumos de alta potencia.

Desde el punto de vista funcional, esta pantalla implementa un mecanismo de actualización periódica mediante solicitudes HTTP de tipo GET al endpoint energía. A través de estas consultas se obtienen las variables asociadas al estado de los distintos equipos, lo que permite su monitoreo en tiempo real.

4.2. Ensayos sobre la aplicación

En esta etapa se realizaron ensayos sobre la aplicación móvil de forma independiente, con el objetivo de validar su funcionamiento antes de su integración con el sistema embebido.

Para ello, se utilizó el entorno de desarrollo provisto por Ionic mediante el comando `ionic serve`, que permite ejecutar la aplicación en un navegador web. Este entorno facilitó el acceso a herramientas de depuración, tales como la simulación de distintos tamaños de pantalla y la visualización de mensajes en la consola.

La figura 4.2 muestra el entorno de ejecución utilizado durante los ensayos.

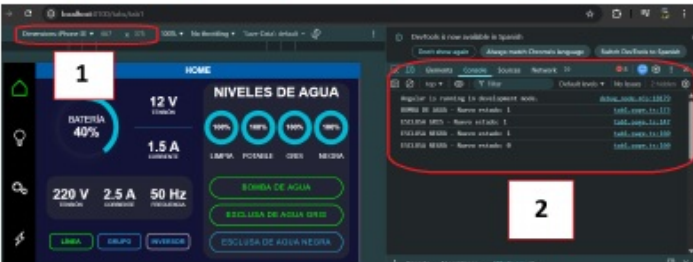


FIGURA 4.2. Entorno de ejecución y depuración de la aplicación.

En el sector identificado como cuadro 1, se empleó la funcionalidad de simulación de distintos tamaños de pantalla, lo que permitió evaluar el comportamiento de la interfaz frente a diferentes resoluciones.

Por otro lado, en el cuadro 2 se encuentra la consola del navegador, donde se visualizaron mensajes de depuración generados mediante instrucciones como `console.log()`. Estos mensajes incluyeron valores de variables y estructuras de datos en formato JSON, lo que facilitó el seguimiento del estado interno de la aplicación y la detección de errores durante su ejecución.

El procedimiento de ensayo consistió en navegar entre las distintas pestañas de la aplicación, accionar los controles disponibles y observar el comportamiento de la interfaz. En paralelo, se analizaron los mensajes generados en la consola del navegador mediante instrucciones de depuración incorporadas en el código.

Adicionalmente, se implementó un esquema de prueba basado en la modificación manual de variables internas (*hardcodeo*), con el fin de simular distintos estados del sistema. Esto permitió evaluar la forma en que la aplicación representa en pantalla valores correspondientes a diferentes condiciones de operación.

Como resultado, se verificó que la navegación entre pantallas se realizaba correctamente, que los controles respondían a las acciones del usuario y que los indicadores visuales reflejaban de manera coherente los valores asignados durante las pruebas.

Asimismo, se comprobó que la interfaz mantenía un comportamiento consistente frente a distintos tamaños de pantalla, sin presentar errores de visualización ni desajustes en la distribución de los elementos.

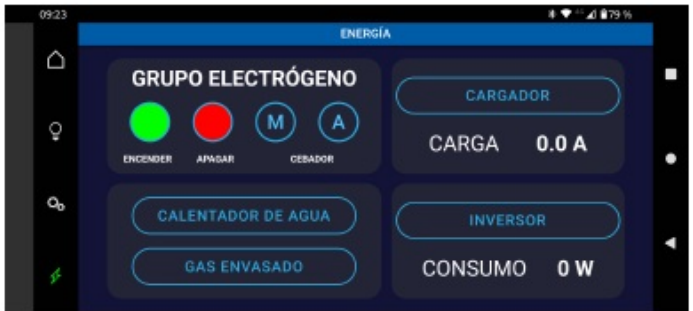


FIGURA 3.13. Pestaña energía de la aplicación.

La interfaz agrupa los controles según el tipo de sistema. En primer lugar, se dispone del control del grupo electrógeno, que permite su encendido y apagado, así como la selección del modo de operación del cebador.

Por otro lado, se incluyen los controles asociados al cargador y al inversor, junto con la visualización de variables relevantes como la corriente de carga y la potencia suministrada.

Además, se incorporan controles correspondientes a otros sistemas de consumo energético, como el calentador de agua y el sistema de gas envasado de alta potencia. Esta integración permite centralizar el manejo de distintos dispositivos dentro de una misma interfaz.

Las acciones de control se envían mediante solicitudes HTTP de tipo POST al endpoint pulsadores, donde se transmite la identificación del equipo y el estado requerido.

De esta manera, la pestaña de energía permite supervisar y gestionar los distintos subsistemas energéticos desde un único punto de acceso, en coherencia con la arquitectura de control definida para el sistema.

3.3.5. Aplicación: generación de instalador para Android

Una vez finalizado el desarrollo de la aplicación, fue necesario generar un instalador que permita su ejecución en dispositivos móviles con sistema operativo Android. Para ello, se utilizó la herramienta Capacitor, que posibilita la adaptación de aplicaciones desarrolladas con tecnologías web a un entorno nativo.

Capacitor actúa como una capa de integración entre la aplicación desarrollada con Ionic y el sistema operativo del dispositivo. A través de esta herramienta, la aplicación se empaqueta en una estructura compatible con Android. Además, brinda acceso a funcionalidades propias del dispositivo.

En este contexto, se emplearon herramientas provistas por Capacitor para la gestión de solicitudes HTTP desde la aplicación, y permitir así un control más directo sobre la comunicación con el servidor del ESP32. Asimismo, se utilizaron utilidades para la generación y configuración de los recursos gráficos de la aplicación, como los íconos y elementos visuales requeridos por el sistema Android.

A partir de estos ensayos, se concluyó que la aplicación presentaba un funcionamiento adecuado a nivel de interfaz y lógica interna, lo que permitió avanzar hacia su integración con el sistema basado en ESP32.

4.3. Ensayos sobre el sistema

En esta etapa se realizaron ensayos de integración entre la aplicación móvil y el sistema basado en ESP32, con el objetivo de validar la comunicación entre ambos y el correcto funcionamiento del servidor implementado en el microcontrolador.

Para ello, se utilizó el entorno de ejecución de la aplicación en el navegador web, junto con el monitor serie del IDE Arduino, empleado para visualizar mensajes de depuración generados por el firmware del ESP32.

El montaje de ensayo consistió en conectar el ESP32 a una red Wi-Fi local, compartida con la computadora en la que se ejecutaba la aplicación.

Al establecer la conexión, el microcontrolador informó su dirección IP a través del monitor serie, lo que permitió acceder a los endpoints del servidor desde la aplicación.

El procedimiento de prueba incluyó el acceso a los distintos endpoints mediante solicitudes HTTP de tipo GET, así como el envío de comandos a través de solicitudes POST generadas desde la interfaz.

Con el fin de evaluar la comunicación, se implementó en el firmware una función temporal que asignaba valores aleatorios a las variables contenidas en las estructuras JSON. Estos valores eran informados mediante mensajes en el monitor serie.

A partir de este esquema, se compararon los valores reportados por el ESP32 con los recibidos por la aplicación, y se verificó su coincidencia junto con su correcta representación en la interfaz.

La figura 4.3 muestra un ejemplo del archivo JSON recibido desde el endpoint home, que contiene tanto variables del sistema como estados de los pulsadores.

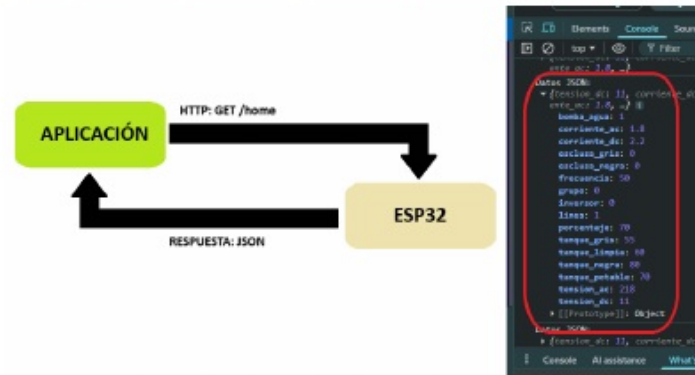


FIGURA 4.3. Archivo JSON recibido en la pestaña home.

A partir de este proceso, el trabajo fue importado en el entorno de desarrollo Android Studio. Este software permite gestionar la compilación de la aplicación, así como configurar parámetros propios del sistema Android, tales como permisos, versión mínima del sistema operativo y opciones de despliegue.

Como resultado de la compilación, se obtiene un archivo en formato APK, que constituye el formato estándar de distribución de aplicaciones para el sistema operativo Android. Este archivo contiene todos los recursos necesarios para la instalación de la aplicación, tales como el código ejecutable, las bibliotecas y los archivos de configuración.

La instalación del APK puede realizarse directamente sobre un dispositivo móvil mediante distintas alternativas. En este trabajo se utilizó la herramienta ADB (Android Debug Bridge) [36], que permite la transferencia e instalación del archivo desde una computadora hacia el dispositivo a través de una conexión USB. Una vez instalado, el usuario puede ejecutar la aplicación de forma independiente, sin necesidad de herramientas de desarrollo adicionales.

Por otra parte, se evaluó el envío de comandos desde la aplicación hacia el ESP32 mediante solicitudes POST. La figura 4.4 muestra el mensaje recibido por el microcontrolador al accionar el pulsador correspondiente a la bomba de agua.

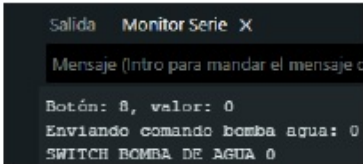


FIGURA 4.4. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.

En este caso, el identificador del comando y el valor asociado fueron correctamente interpretados por el sistema, lo que dio lugar al envío de la instrucción correspondiente hacia el módulo PRC10.

Asimismo, se verificó la respuesta del servidor ante estas solicitudes. La figura 4.5 presenta el contenido de la respuesta enviada por el ESP32, donde se confirma el comando ejecutado junto con el nuevo estado asignado.



FIGURA 4.5. Respuesta del servidor ante una solicitud de control.

Como resultado, se comprobó que los datos intercambiados entre la aplicación y el ESP32 coincidían de manera consistente, que los comandos enviados eran correctamente procesados y que las respuestas del servidor reflejaban el estado actualizado del sistema.

A partir de estos ensayos, se concluyó que la comunicación entre la aplicación y el microcontrolador resultó confiable, lo que permitió validar el funcionamiento del servidor y avanzar hacia la etapa de pruebas con equipos reales.

4.4. Ensayos de laboratorio

En esta etapa se realizaron ensayos con equipos reales de la red de domótica de Enertik, con el objetivo de validar el funcionamiento del sistema en condiciones representativas de operación.

4.4.1. Prototipo experimental

Para realizar los primeros ensayos, se decidió realizar un primer prototipo del equipo. Para ello, se utilizó como base un PCB (*Printed Circuit Board*) perforado de tipo experimental, sobre el que se soldaron diversos conectores, cableados y componentes de forma manual.

La figura 4.6 muestra el montaje del prototipo utilizado para los ensayos iniciales.

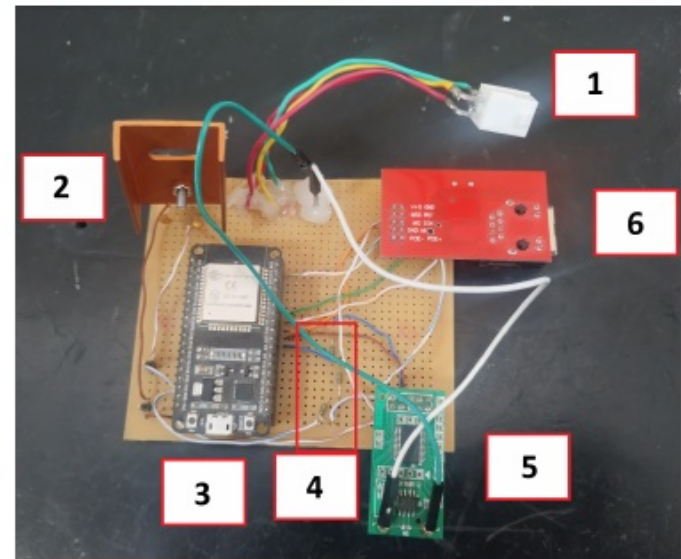


FIGURA 4.6. Prototipo experimental del equipo.

Como se puede apreciar, el prototipo cuenta con cada uno de los módulos explicados en la sección 3.1, donde:

- Componente 1: conector de cuatro hilos. Este conector es el estándar empleado en la red de domótica de Enertik. A través de él se energizan todos los componentes del PCB experimental y se accede a la comunicación CAN de la red.
- Componente 2: regulador de tensión 7805 [30]. Este dispositivo se encarga de suministrar una tensión estable de 5 V. Dado que la tensión proveniente del motorhome es de 12 V, fue necesaria su utilización para adecuar los niveles de alimentación a los requeridos por los módulos.
- Componente 3: módulo ESP32. Es el microcontrolador principal de la placa y contiene el firmware de control del sistema. En este prototipo se alimentó a través del regulador 7805, ya que la placa de desarrollo dispone internamente de reguladores que reducen la tensión a 3,3 V. Para el desarrollo final fue necesaria la incorporación de un regulador adicional dedicado al ESP32.

Capítulo 4

Ensayos y resultados

En este capítulo se explica cómo fueron los ensayos que se realizaron sobre el sistema y qué resultados se obtuvieron a partir de ellos.

4.1. Ensayos sobre la aplicación

En esta etapa se realizaron ensayos sobre la aplicación móvil de forma independiente, con el objetivo de validar su funcionamiento antes de su integración con el sistema embebido.

Para ello, se utilizó el entorno de desarrollo provisto por Ionic mediante el comando `ionic serve`, que permite ejecutar la aplicación en un navegador web. Este entorno facilitó el acceso a herramientas de depuración, tales como la simulación de distintos tamaños de pantalla y la visualización de mensajes en la consola.

La figura 4.1 muestra el entorno de ejecución utilizado durante los ensayos.



FIGURA 4.1. Entorno de ejecución y depuración de la aplicación.

En el sector identificado como cuadro 1, se empleó la funcionalidad de simulación de distintos tamaños de pantalla, lo que permitió evaluar el comportamiento de la interfaz frente a diferentes resoluciones.

- Componente 4: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceptor.
- Componente 5: módulo transceptor. Se trata de una placa comercial provista por Enertik. En este prototipo se decidió incorporar un conector en el PCB experimental para facilitar su montaje y desmontaje de forma segura.
- Componente 6: módulo W5100. De manera similar al módulo transceptor, se utilizó un conector compatible que permite su montaje y desmontaje de forma sencilla y segura.

Este prototipo permitió validar el funcionamiento general del sistema y realizar pruebas iniciales de integración entre los distintos módulos de hardware.

4.4.2. Ensayos de integración del sistema completo

La figura 4.7 muestra el módulo PRC10 utilizado durante los ensayos. Este equipo fue separado y destinado específicamente a este desarrollo.



FIGURA 4.7. Módulo PRC10 utilizado en los ensayos.

Para la realización de las pruebas, se utilizó un maletín provisto por Enertik que integra los equipos comerciales empleados en aplicaciones de motorhomes. Este conjunto reproduce la configuración real del sistema, tanto a nivel de alimentación como de comunicación.

La figura 4.8 muestra dicho entorno, donde los dispositivos se encuentran interconectados mediante un cableado de cuatro conductores. En ella se identifican el prototipo desarrollado (cuadro 1) y el módulo PRC10 (cuadro 2).

El sistema dispone de dos entradas de alimentación: una fuente de 12 V en corriente continua, que representa la batería, y una entrada de corriente alterna que simula las fuentes de línea, grupo electrógeno e inversor. Como carga, se utilizaron luces testigo que permiten visualizar el accionamiento de los distintos circuitos.

Por otro lado, en el cuadro 2 se encuentra la consola del navegador, donde se visualizaron mensajes de depuración generados mediante instrucciones como `console.log()`. Estos mensajes incluyeron valores de variables y estructuras de datos en formato JSON, lo que facilitó el seguimiento del estado interno de la aplicación y la detección de errores durante su ejecución.

El procedimiento de ensayo consistió en navegar entre las distintas pestañas de la aplicación, accionar los controles disponibles y observar el comportamiento de la interfaz. En paralelo, se analizaron los mensajes generados en la consola del navegador mediante instrucciones de depuración incorporadas en el código.

Adicionalmente, se implementó un esquema de prueba basado en la modificación manual de variables internas (*hardcodeo*), con el fin de simular distintos estados del sistema. Esto permitió evaluar la forma en que la aplicación representa en pantalla valores correspondientes a diferentes condiciones de operación.

Como resultado, se verificó que la navegación entre pantallas se realizaba correctamente, que los controles respondían a las acciones del usuario y que los indicadores visuales reflejaban de manera coherente los valores asignados durante las pruebas.

Asimismo, se comprobó que la interfaz mantenía un comportamiento consistente frente a distintos tamaños de pantalla, sin presentar errores de visualización ni desajustes en la distribución de los elementos.

A partir de estos ensayos, se concluyó que la aplicación presentaba un funcionamiento adecuado a nivel de interfaz y lógica interna, lo que permitió avanzar hacia su integración con el sistema basado en ESP32.

4.2. Ensayos sobre el sistema

En esta etapa se realizaron ensayos de integración entre la aplicación móvil y el sistema basado en ESP32, con el objetivo de validar la comunicación entre ambos y el correcto funcionamiento del servidor implementado en el microcontrolador.

Para ello, se utilizó el entorno de ejecución de la aplicación en el navegador web, junto con el monitor serie del IDE Arduino, empleado para visualizar mensajes de depuración generados por el firmware del ESP32.

El montaje de ensayo consistió en conectar el ESP32 a una red Wi-Fi local, compartida con la computadora en la que se ejecutaba la aplicación.

Al establecer la conexión, el microcontrolador informó su dirección IP a través del monitor serie, lo que permitió acceder a los endpoints del servidor desde la aplicación.

El procedimiento de prueba incluyó el acceso a los distintos endpoints mediante solicitudes HTTP de tipo GET, así como el envío de comandos a través de solicitudes POST generadas desde la interfaz.

Con el fin de evaluar la comunicación, se implementó en el firmware una función temporal que asignaba valores aleatorios a las variables contenidas en las estructuras JSON. Estos valores eran informados mediante mensajes en el monitor serie.



FIGURA 4.8. Equipos Enertik utilizados en los ensayos.

Para la integración con el ecosistema Victron, se emplearon un módulo Cerbo GX y un equipo MultiPlus-II 12/3000/120-32 230V [37], tal como se muestra en la figura 4.9.

El procedimiento de ensayo consistió en instalar la aplicación en un dispositivo móvil con sistema operativo Android, establecer la conexión a la red Wi-Fi generada por el ESP32 y evaluar el comportamiento de cada una de las pestañas.

En la pestaña principal se accionaron los pulsadores y se verificó la respuesta de las luces testigo asociadas. Asimismo, se aplicaron variaciones sobre los sensores de nivel de los tanques, con el fin de simular distintos estados y comprobar la actualización de los indicadores en la interfaz.

Este procedimiento se replicó en las demás pestañas, donde se evaluaron los distintos controles implementados y su efecto sobre los dispositivos físicos del sistema.

En relación con la comunicación con los equipos Victron, se utilizó la pestaña energía para controlar el equipo MultiPlus.

Al activar el Multiplus en modo inversor, el equipo comenzó a convertir la tensión de batería en una señal alterna de 220 V. Como carga, se conectó un soldador de tipo *cautín* de alta potencia, lo que permitió observar la variación de la potencia indicada en la aplicación.

A partir de este esquema, se compararon los valores reportados por el ESP32 con los recibidos por la aplicación, y se verificó su coincidencia junto con su correcta representación en la interfaz.

La figura 4.2 muestra un ejemplo del archivo JSON recibido desde el endpoint home, que contiene tanto variables del sistema como estados de los pulsadores.



FIGURA 4.2. Archivo JSON recibido en la pestaña home.

Por otra parte, se evaluó el envío de comandos desde la aplicación hacia el ESP32 mediante solicitudes POST. La figura 4.3 muestra el mensaje recibido por el microcontrolador al accionar el pulsador correspondiente a la bomba de agua.

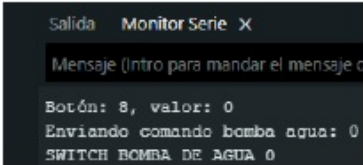


FIGURA 4.3. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.

En este caso, el identificador del comando y el valor asociado fueron correctamente interpretados por el sistema, lo que dio lugar al envío de la instrucción correspondiente hacia el módulo PRC10.

Asimismo, se verificó la respuesta del servidor ante estas solicitudes. La figura 4.4 presenta el contenido de la respuesta enviada por el ESP32, donde se confirma el comando ejecutado junto con el nuevo estado asignado.

Como resultado, se comprobó que los datos intercambiados entre la aplicación y el ESP32 coincidían de manera consistente, que los comandos enviados eran correctamente procesados y que las respuestas del servidor reflejaban el estado actualizado del sistema.



FIGURA 4.9. Equipo Victron MultiPlus-II y módulo Cerbo GX.

Por otro lado, se conectó el Multiplus a la red domiciliaria y se lo configuró en modo cargador. Mediante el uso de una pinza amperométrica, se verificó que el valor de corriente de carga mostrado en la aplicación resultaba coherente con la medición real.

Como criterio de validación, se consideró satisfactorio que los estados de los actuadores coincidieran con las acciones realizadas desde la aplicación, y que las variables medidas presentaran valores coherentes con las lecturas obtenidas mediante instrumentos externos.

Los resultados obtenidos indicaron que el sistema respondió correctamente a las acciones de control, tanto en los dispositivos de la red de Enertik como en los equipos del ecosistema Victron. Las magnitudes eléctricas visualizadas en la aplicación resultaron consistentes con las mediciones realizadas.

Adicionalmente, se realizó una prueba en un motorhome real durante una visita a la fábrica, donde se evaluaron los controles en condiciones reales de operación. En esta instancia, el sistema presentó un comportamiento adecuado y consistente con los ensayos de laboratorio.

No obstante, no fue posible validar la comunicación con equipos Victron en el vehículo, debido a la ausencia de este equipamiento en la unidad ensayada. Esta verificación queda planteada como trabajo futuro.

Finalmente, debido a restricciones de la empresa fabricante, no se dispuso de registros fotográficos de los ensayos realizados en el motorhome.

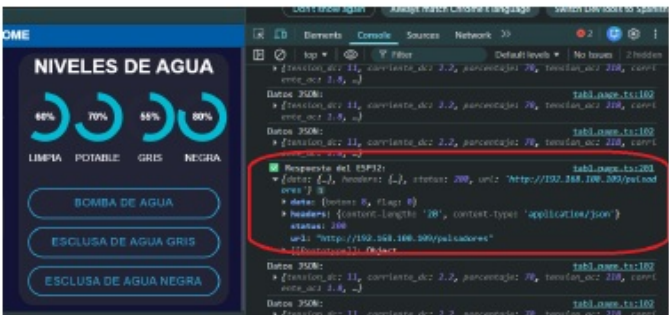


FIGURA 4.4. Respuesta del servidor ante una solicitud de control.

A partir de estos ensayos, se concluyó que la comunicación entre la aplicación y el microcontrolador resultó confiable, lo que permitió validar el funcionamiento del servidor y avanzar hacia la etapa de pruebas con equipos reales.

4.3. Ensayos de laboratorio

En esta etapa se realizaron ensayos con equipos reales de la red de domótica de Enertik, con el objetivo de validar el funcionamiento del sistema en condiciones representativas de operación.

La figura 4.5 muestra el módulo PRC10 utilizado durante los ensayos. Este equipo fue separado y destinado específicamente a este desarrollo.



FIGURA 4.5. Módulo PRC10 utilizado en los ensayos.

Capítulo 5

Conclusiones

En este capítulo se presentan las conclusiones obtenidas de la ejecución de este trabajo y, además, se proponen perspectivas que podrían ser implementadas a futuro.

5.1. Conclusiones generales

En primer lugar, se destaca que fue posible desarrollar con éxito un dispositivo capaz de comunicarse tanto con la red de domótica de Enertik como con el ecosistema Victron.

Este resultado resulta de gran relevancia, ya que habilita la futura integración de equipos Victron dentro de la red de domótica del motorhome, amplía las capacidades del sistema y permite una mayor flexibilidad en su implementación.

Asimismo, se logró el desarrollo de una aplicación funcional que permite interactuar con el sistema de domótica. Esta aplicación es capaz de replicar la mayoría de las funcionalidades disponibles en la pantalla HMI del motorhome y brinda a los usuarios una alternativa adicional para el control del sistema.

A continuación, se destacan los principales aspectos del trabajo:

- Se desarrolló con éxito un dispositivo capaz de comunicarse con ambos ecosistemas de manera confiable.
- Se diseñó un PCB de tamaño reducido, orientado a su futura implementación como producto comercial.
- El equipo se integra a la red del motorhome mediante el cableado estándar de cuatro conductores, lo que facilita su instalación.
- Se implementó una arquitectura de firmware basada en múltiples tareas, lo que permite un funcionamiento organizado y eficiente del sistema.
- La aplicación desarrollada funciona correctamente en dispositivos Android, lo que amplía las posibilidades de uso por parte de los usuarios.
- La aplicación replica la mayoría de las funcionalidades de la pantalla HMI, lo que reduce la dependencia de esta para el control del sistema.
- Se validó el funcionamiento del sistema tanto en entorno de laboratorio como en condiciones reales de operación.
- La solución presenta un diseño escalable, lo que permite la incorporación de nuevas funcionalidades y dispositivos en el futuro.

Para la realización de las pruebas, se utilizó un maletín provisto por Enertik que integra los equipos comerciales empleados en aplicaciones de motorhomes. Este conjunto reproduce la configuración real del sistema, tanto a nivel de alimentación como de comunicación.

La figura 4.6 muestra dicho entorno, donde los dispositivos se encuentran interconectados mediante un cableado de cuatro conductores. En ella se identifican el prototipo desarrollado (cuadro 1) y el módulo PRC10 (cuadro 2).



FIGURA 4.6. Equipos Enertik utilizados en los ensayos.

El sistema dispone de dos entradas de alimentación: una fuente de 12 V en corriente continua, que representa la batería, y una entrada de corriente alterna que simula las fuentes de línea, grupo electrógeno e inversor. Como carga, se utilizaron luces testigo que permiten visualizar el accionamiento de los distintos circuitos.

Para la integración con el ecosistema Victron, se emplearon un módulo Cerbo GX y un equipo MultiPlus-II 12/3000/120-32 230V [37], tal como se muestra en la figura 4.7.

El procedimiento de ensayo consistió en instalar la aplicación en un dispositivo móvil con sistema operativo Android, establecer la conexión a la red Wi-Fi generada por el ESP32 y evaluar el comportamiento de cada una de las pestañas.

- Se emplearon protocolos de comunicación estándar, lo que favorece la interoperabilidad con otros sistemas.

5.2. Próximos pasos

Como líneas de trabajo a futuro, se consideran diversas mejoras y extensiones del sistema desarrollado.

En primer lugar, durante los ensayos en vehículos reales, los fabricantes de motorhomes señalaron que la aplicación requiere la conexión a la red Wi-Fi generada por el ESP32, lo que implica que el teléfono celular pierde el acceso a Internet mientras se utiliza el sistema. Si bien esta situación no resulta crítica, se analizaron distintas alternativas para su resolución.

Una posible solución consiste en modificar el firmware del ESP32 para que se conecte a una red Wi-Fi existente. De esta manera, la aplicación, al conectarse a la misma red, podría acceder a los endpoints del servidor mediante la dirección IP asignada por el módem.

Otra alternativa consiste en migrar el protocolo de comunicación entre la aplicación y el ESP32. En este sentido, se podría implementar una comunicación mediante Bluetooth en reemplazo de la conexión Wi-Fi. Ambas opciones permitirían mantener la conectividad a Internet en el dispositivo móvil y, al mismo tiempo, conservar el control del sistema sin inconvenientes.

Por otra parte, se considera necesaria la adaptación de la aplicación para su ejecución en dispositivos iOS. Esta tarea no se incluyó dentro del alcance del presente trabajo.

En la actualidad, los sistemas operativos predominantes en dispositivos móviles son Android e iOS. La compatibilidad con ambos sistemas permitiría ampliar el alcance de la solución, al cubrir una mayor cantidad de usuarios potenciales.

Relacionado con este aspecto, también resulta necesario realizar modificaciones que permitan distribuir la aplicación a través de tiendas oficiales, como Play Store en Android y App Store en iOS.

En la actualidad, la aplicación solo puede instalarse mediante un archivo de tipo APK. Sin embargo, para su implementación a nivel comercial, resulta conveniente que los usuarios puedan acceder a ella a través de estas plataformas, lo que facilitaría su distribución, instalación y actualización.



FIGURA 4.7. Equipo Victron MultiPlus-II y módulo Cerbo GX.

En la pestaña principal se accionaron los pulsadores y se verificó la respuesta de las luces testigo asociadas. Asimismo, se aplicaron variaciones sobre los sensores de nivel de los tanques, con el fin de simular distintos estados y comprobar la actualización de los indicadores en la interfaz.

Este procedimiento se replicó en las demás pestañas, donde se evaluaron los distintos controles implementados y su efecto sobre los dispositivos físicos del sistema.

En relación con la comunicación con los equipos Victron, se utilizó la pestaña energía para controlar el equipo MultiPlus.

Al activar el Multiplus en modo inversor, el equipo comenzó a convertir la tensión de batería en una señal alterna de 220 V. Como carga, se conectó un soldador de tipo *cautín* de alta potencia, lo que permitió observar la variación de la potencia indicada en la aplicación.

Por otro lado, se conectó el Multiplus a la red domiciliaria y se lo configuró en modo cargador. Mediante el uso de una pinza amperométrica, se verificó que el valor de corriente de carga mostrado en la aplicación resultaba coherente con la medición real.

Como criterio de validación, se consideró satisfactorio que los estados de los actuadores coincidieran con las acciones realizadas desde la aplicación, y que las variables medidas presentaran valores coherentes con las lecturas obtenidas mediante instrumentos externos.

Los resultados obtenidos indicaron que el sistema respondió correctamente a las acciones de control, tanto en los dispositivos de la red de Enertik como en los equipos del ecosistema Victron. Las magnitudes eléctricas visualizadas en la aplicación resultaron consistentes con las mediciones realizadas.

Adicionalmente, se realizó una prueba en un motorhome real durante una visita a la fábrica, donde se evaluaron los controles en condiciones reales de operación. En esta instancia, el sistema presentó un comportamiento adecuado y consistente con los ensayos de laboratorio.

Bibliografía

[1] Enertik Argentina. Última consulta: 2025-03-14. URL: <https://enertik.com/ar/>.

[2] Victron Energy. Última consulta: 2025-03-14. URL: <https://www.victronenergy.com.es/>.

[3] Enertik Argentina. *PRC10: manual para el usuario*.

[4] Espressif. Última consulta: 2025-03-21. URL: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html>.

[5] Cerbo GX. Última consulta: 2025-03-16. URL: <https://www.victronenergy.com/es/communication-centres/cerbo-gx>.

[6] CAN Bus Explained - A Simple Intro [2025]. Última consulta: 2025-03-16. URL: <https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial>.

[7] Modbus: Qué es y cómo funciona. Última consulta: 2025-03-19. URL: <https://www.cursosaula21.com/modbus-que-es-y-como-functiona/>.

[8] Overview of HTTP. Última consulta: 2025-03-19. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>.

[9] TCP/IP Overview. Última consulta: 2025-03-25. URL: <https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13769-5.html>.

[10] Manual de GX - Modbus TCP. Última consulta: 2025-03-15. URL: https://www.victronenergy.com/live/ccgx:modbustcp_faq.

[11] ¿Qué es una Petición HTTP? Última consulta: 2025-03-29. URL: <https://kinsta.com/es/blog/que-es-una-peticion-http/>.

[12] W5100. Última consulta: 2025-03-19. URL: https://cdn.sparkfun.com/assets/e/2/f/c/2/W5100_Datasheet_v1_1_6.pdf.

[13] MCP2551. Última consulta: 2025-03-19. URL: <https://ww1.microchip.com/downloads/en/devicedoc/20001667g.pdf>.

[14] Serial Communication. Última consulta: 2025-03-29. URL: <https://learn.sparkfun.com/tutorials/serial-communication/all>.

[15] Serial Peripheral Interface (SPI). Última consulta: 2025-03-29. URL: <https://learn.sparkfun.com/tutorials/serial-peripheral-interface-spi/all>.

[16] I2C. Última consulta: 2025-04-08. URL: <https://learn.sparkfun.com/tutorials/i2c>.

[17] Pulse Width Modulation (PWM): How it Works and Why it's Essential in Electronics. Última consulta: 2025-04-02. URL: <https://www.sameskydevices.com/blog/pulse-width-modulation-pwm-how-it-works-and-why-its-essential-in-electronics>.

[18] PIC18F66K80 FAMILY. Última consulta: 2025-03-19. URL: <https://ww1.microchip.com/downloads/en/DeviceDoc/PIC18F66K80%20FAMILY%20Enhanced%20Flash%20MCU%20with%20ECAN%20XLP%20Technology%2030009977G.pdf>.

[19] Arduino. Última consulta: 2025-03-25. URL: <https://docs.arduino.cc/software/ide-v1/tutorials/PortableIDE/>.

No obstante, no fue posible validar la comunicación con equipos Victron en el vehículo, debido a la ausencia de este equipamiento en la unidad ensayada. Esta verificación queda planteada como trabajo futuro.

Finalmente, debido a restricciones de la empresa fabricante, no se dispuso de registros fotográficos de los ensayos realizados en el motorhome.

- [20] Github: modbus-esp8266. Última consulta: 2025-03-25. URL: <https://github.com/emelianov/modbus-esp8266>.
- [21] FreeRTOS. Última consulta: 2025-03-19. URL: <https://www.freertos.org/>.
- [22] Introduction to Ionic. Última consulta: 2025-03-19. URL: <https://ionicframework.com/docs>.
- [23] Angular. Última consulta: 2025-03-25. URL: <https://angular.dev/>.
- [24] What is TypeScript? Última consulta: 2025-03-29. URL: <https://www.typescriptlang.org/>.
- [25] HTML: HyperText Markup Language. Última consulta: 2025-03-29. URL: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [26] CSS: Cascading Style Sheets. Última consulta: 2025-03-29. URL: <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [27] Capacitor web. Última consulta: 2025-03-25. URL: <https://capacitorjs.com/>.
- [28] Android Studio. Última consulta: 2025-03-25. URL: <https://developer.android.com/studio?hl=es-419>.
- [29] ¿Qué es un conector RJ45? Todo lo que necesitas saber. Última consulta: 2025-04-15. URL: <https://seetronic.com/es/blog/what-is-an-rj45-connector/>.
- [30] LM340, LM340A and LM7805 Family Wide VIN 1.5-A Fixed Voltage Regulators. Última consulta: 2025-03-29. URL: <https://www.ti.com/lit/ds/symlink/lm340.pdf>.
- [31] Altium Designer. Última consulta: 2025-03-29. URL: <https://www.altium.com/es/altium-designer>.
- [32] AP63200/AP63201/AP63203/AP63205. Última consulta: 2025-03-29. URL: <https://4donline.ihs.com/images/VipMasterIC/IC/DIOD/DIOD-S-A0007089856/DIOD-S-A0007089856-1.pdf?hkey=CECEF36DEECD6468708AAF2E19C0C6>.
- [33] What is a gerber file and how are they used in the PCB fabrication process. Última consulta: 2025-03-29. URL: <https://resources.altium.com/es/p/what-gerber-file-pcb-fabrication-process>.
- [34] What is RingBuffer? Última consulta: 2025-03-30. URL: <https://medium.com/@kolheankita15/what-is-ringbuffer-7b6b808f33e0>.
- [35] Introducing JSON. Última consulta: 2025-04-02. URL: <https://www.json.org/json-en.html>.
- [36] Android Debug Bridge (adb). Última consulta: 2025-04-02. URL: <https://developer.android.com/tools/adb?hl=es-419>.
- [37] Victron MultiPlus-II 12/3000/120-32 230V. Última consulta: 2025-04-02. URL: <https://enertik.com/ar/tienda/inversores/inversores-cargadores/sin-regulador-solar/victron-multiplus-ii-123000120-32-230v/>.